

Análise Experimental do Desempenho da Hierarquia de Memória e Subsistemas de I/O em Processador Híbrido Intel de 13ª Geração

Pedro Coutinho da Silva¹

¹CESAR School

Recife, PE, Brasil

Orientador: Prof. Ronierison Maciel

`pcs4@cesar.school`

Resumo. Este artigo apresenta uma análise experimental do comportamento de desempenho da hierarquia de memória, dos subsistemas de I/O e do paralelismo de CPU em uma plataforma equipada com o processador Intel Core i5-13420H de 13ª geração, pertencente à microarquitetura Raptor Lake, com 8 GiB de RAM DDR4 instalados (7,6 GiB úteis ao sistema). Os experimentos foram conduzidos em ambiente de laboratório sob o sistema operacional Ubuntu 24.04 LTS executado sobre WSL2, empregando ferramentas como *mbw*, *fio*, *stress-ng* e scripts de carga sintética. Os resultados demonstram que o gargalo primário do sistema avaliado é a largura de banda de memória principal, o que se manifesta na degradação de desempenho ao escalar o paralelismo além de quatro threads. A análise dos barramentos PCIe revelou subutilização da capacidade do controlador NVMe, que opera em Gen 3x4 apesar de possuir capacidade para Gen 4x4. Os achados corroboram os princípios teóricos da Lei de Amdahl e da localidade de referência, evidenciando a necessidade de considerar a arquitetura de memória como fator central na avaliação de plataformas computacionais.

1. Introdução

A avaliação de desempenho de sistemas computacionais exige uma abordagem que transcende a análise isolada de componentes. A interação entre processador, hierarquia de memória e subsistemas de entrada e saída determina o comportamento real de uma plataforma sob cargas de trabalho diversas. Tal perspectiva sistêmica ganha relevância especial com a disseminação de processadores que combinam núcleos de alto desempenho e núcleos de alta eficiência energética em um único *die*, como ocorre na microarquitetura Intel Raptor Lake.

Este trabalho investiga experimentalmente o impacto da hierarquia de memória no desempenho de um sistema equipado com o processador Intel Core i5-13420H, que adota uma topologia heterogênea composta por quatro *Performance-cores* (P-cores) e quatro *Efficient-cores* (E-cores), totalizando 12 threads lógicas. A hipótese central é que, em cargas de trabalho intensivas em memória, a largura de banda da memória principal constitui o gargalo dominante, tornando a adição de unidades de processamento ineficaz ou até prejudicial ao desempenho agregado.

O artigo está organizado da seguinte forma. A Seção 2 discute os trabalhos relacionados. A Seção 3 apresenta a fundamentação teórica sobre hierarquia de memória,

arquitetura de processadores híbridos e barramentos PCIe. A Seção 4 descreve o ambiente experimental e as ferramentas empregadas. A Seção 5 apresenta e discute os resultados obtidos. Por fim, a Seção 6 sintetiza as conclusões e aponta direções para investigações futuras.

2. Trabalhos Relacionados

A caracterização experimental de subsistemas de memória possui tradição consolidada na literatura de avaliação de desempenho. O benchmark STREAM [McCalpin 1995] estabeleceu a medição de largura de banda sustentável como métrica central para a avaliação de plataformas, demonstrando que a vazão de memória cresce em ritmo inferior ao poder de processamento ao longo das gerações de hardware — desequilíbrio que ficou conhecido como *memory wall*. O conjunto LMBench [McVoy and Staelin 1996] ampliou essa abordagem ao propor microbenchmarks portáteis de latência e largura de banda para os diversos níveis da hierarquia, abrangendo caches, memória principal e operações do sistema operacional.

No plano analítico, o modelo Roofline [Williams et al. 2009] formalizou a distinção entre cargas limitadas por computação (*compute-bound*) e por memória (*memory-bound*), relacionando a intensidade aritmética da aplicação aos tetos de desempenho da máquina. Essa classificação orienta diretamente o diagnóstico de gargalos realizado neste trabalho.

O presente estudo se apoia nessas metodologias consolidadas, mas as aplica a um cenário ainda pouco caracterizado na literatura: uma plataforma híbrida recente, com P-cores e E-cores, operando sob a camada de virtualização do WSL2 — configuração cada vez mais comum em ambientes de desenvolvimento e ensino, na qual a mediação do hipervisor altera aspectos observáveis do subsistema de I/O, como a semântica das interrupções.

3. Fundamentação Teórica

3.1. Hierarquia de Memória e Localidade de Referência

A hierarquia de memória é um dos pilares da arquitetura de computadores moderna. Organiza os diferentes níveis de armazenamento — registradores, caches L1, L2 e L3, memória principal DRAM e armazenamento secundário — segundo uma relação inversa entre capacidade e velocidade de acesso [Patterson and Hennessy 2021]. O princípio da localidade de referência fundamenta a eficácia dessa hierarquia: programas tendem a acessar, em um intervalo curto de tempo, as mesmas posições de memória (localidade temporal) ou posições adjacentes (localidade espacial) [Hennessy and Patterson 2017].

Quando um dado requisitado pelo processador não está presente nos caches (*cache miss*), é necessário buscá-lo em um nível inferior da hierarquia, incorrendo em penalidades de latência que podem superar centenas de ciclos de *clock*. Em sistemas com múltiplos núcleos, esse fenômeno é agravado pela disputa pela largura de banda do barramento de memória, o que pode tornar o desempenho agregado inferior ao esperado pelo escalonamento linear do número de núcleos [McCalpin 1995, Rogers et al. 2009, Diamond et al. 2011].

O conceito de sistema *memory-bound* descreve exatamente essa condição: a CPU permanece ociosa aguardando dados da memória principal, de modo que o aumento do

poder de processamento não se traduz em ganho de desempenho [Williams et al. 2009]. A Lei de Amdahl formaliza esse limite ao expressar que a aceleração máxima de um programa é limitada pela fração sequencial ou pelo gargalo de recursos compartilhados [Amdahl 1967].

3.2. Processadores Híbridos: Arquitetura Intel Raptor Lake

A geração Raptor Lake da Intel, introduzida em 2022, consolidou a arquitetura heterogênea que combina P-cores baseados na microarquitetura Golden Cove com E-cores baseados na microarquitetura Gracemont em um mesmo pacote processador [Intel Corporation 2021, Intel Corporation 2022a]. Os P-cores operam com suporte a *Hyper-Threading*, enquanto os E-cores não possuem SMT, otimizando a densidade de núcleos para tarefas de baixa prioridade e cargas paralelizáveis de menor intensidade computacional.

O i5-13420H, variante voltada para plataformas móveis, apresenta frequência base de 2,1 GHz e frequência *turbo* de até 4,6 GHz, com suporte às extensões vetoriais AVX2 e AVX-VNNI. A hierarquia de cache é composta por 288 KiB de L1d, 192 KiB de L1i, 7,5 MiB de L2 e 12 MiB de cache L3 compartilhado entre todos os núcleos. O escalonador do sistema operacional, em conjunto com o Intel Thread Director, é responsável por distribuir tarefas entre P-cores e E-cores de acordo com a demanda computacional [Intel Corporation 2022b].

3.3. Barramentos PCIe e Subsistemas de I/O

O barramento PCI Express (PCIe) é o padrão dominante para interconexão de dispositivos de alta largura de banda em plataformas modernas [Budruk et al. 2003]. Cada geração do padrão dobra aproximadamente a largura de banda por *lane* em relação à anterior: PCIe Gen 3 oferece 1 GB/s por *lane*, Gen 4 oferece 2 GB/s e Gen 5 alcança 4 GB/s por *lane*. Um controlador NVMe operando em Gen 4x4 dispõe, portanto, de largura de banda bidirecional de 8 GB/s.

A latência de interrupções em sistemas de I/O é mensurada por meio do contador `/proc/interrupts` no Linux, que registra a contagem acumulada de interrupções por linha de IRQ desde a inicialização do sistema [Love 2010]. Em ambientes de virtualização como o WSL2, as interrupções físicas do dispositivo são substituídas por *callbacks* do hipervisor, introduzindo uma camada adicional de indireção entre o dispositivo e o núcleo Linux convidado.

3.4. Memória Virtual e Espaço de Endereçamento

O sistema operacional implementa memória virtual por meio de paginação, separando o espaço de endereçamento lógico do processo do espaço de endereçamento físico disponível [Tanenbaum and Bos 2015]. O campo `VmPeak` do arquivo `/proc/PID/status` registra o pico de espaço de endereçamento virtual reservado pelo processo, enquanto `VmRSS` (*Resident Set Size*) indica a quantidade de memória física efetivamente utilizada. A diferença entre essas métricas ilustra a natureza *lazy* da alocação de memória em sistemas POSIX: reservar espaço de endereçamento não implica consumir RAM física.

4. Metodologia

4.1. Ambiente Experimental

Os experimentos foram conduzidos em uma estação de trabalho equipada com processador Intel Core i5-13420H (13ª geração, microarquitetura Raptor Lake), 8 GiB de RAM DDR4 instalados (7,6 GiB úteis ao sistema), e armazenamento NVMe com controlador SK Hynix (ID PCI 1C5C:1D59). O sistema operacional hospedeiro é o Windows 11, e os testes foram executados sobre o subsistema Ubuntu 24.04.3 LTS com kernel 6.6.87.2-microsoft-standard-WSL2. A GPU dedicada identificada opera via barramento PCIe Gen 4x8 (ID PCI 10DE:28A1).

4.2. Ferramentas e Procedimentos

A Tabela 1 descreve as ferramentas utilizadas em cada etapa experimental.

Tabela 1. Ferramentas empregadas nos experimentos

Ferramenta	Finalidade	Etapa
lscpu, /proc/cpuinfo	Identificação da CPU	1
Cinebench R23	Benchmark mono e multithread	1
Script de paralelismo	Avaliação de <i>speedup</i>	1
mbw	Largura de banda de memória	2
Script de localidade	Localidade espacial de acesso	2
free, vmstat	Monitoramento de RAM e swap	3
fio	Benchmark de I/O em bloco	3
stress-ng	Pressão artificial sobre a memória	3
/proc/PID/status	Memória virtual por processo	3
lspci -vv	Links PCIe (LnkCap e LnkSta)	4
/proc/interrupts	Interrupções durante I/O	4
Workload integrado	Carga sintética CPU/memória/I/O	5

O benchmark `mbw` foi executado para arrays de 16 MiB, 128 MiB e 1.024 MiB, cobrindo a transição entre cache L3 e memória principal. O benchmark `fio` empregou `-direct=1`, `-ioengine=libaio` e `-time_based` para minimizar efeitos de *cache* do sistema de arquivos. A pressão de RAM com `stress-ng` foi configurada para 80% da memória disponível, com contadores de *swap* monitorados via `vmstat` 1 5. A contagem de interrupções foi registrada antes e após a execução de `dd` com transferência de 2 GB em blocos de 1 MiB. Os scripts utilizados e as saídas brutas das medições encontram-se disponíveis em repositório público, garantindo a reprodutibilidade dos experimentos.

5. Resultados e Discussão

5.1. Desempenho da CPU e Escalabilidade de Threads

O Cinebench R23 registrou 2.156,88 eventos/s no modo *single core* e 15.808,60 eventos/s no modo *multi core*, resultando em fator de escala de 7,33x. Considerando 12 *threads*

disponíveis, um escalonamento ideal produziria fator de 12x, o que implica eficiência paralela de apenas 61% no benchmark de renderização.

O teste de paralelismo com o script Python revelou degradação ainda mais acentuada, conforme a Tabela 2. O ponto ótimo foi atingido com 4 *threads* (0,22 s), ao passo que a execução com 12 *threads* resultou em tempo de 0,31 s, 41% superior ao ponto ótimo.

Tabela 2. Escalabilidade de *threads* no teste de paralelismo

Threads	Tempo (s)	Speedup	Eficiência (%)
1	0,45	1,00x	100,0
2	0,27	1,68x	84,2
4	0,22	2,05x	51,3
12	0,31	1,45x	12,1

A explicação reside em dois fatores interligados. Primeiro, cada tarefa aloca aproximadamente 10 MB de matrizes; com 12 processos simultâneos, a demanda agregada de 120 MB excede amplamente o cache L3 de 12 MiB, forçando todos os acessos à memória principal e saturando sua largura de banda. Segundo, a topologia híbrida do i5-13420H introduz heterogeneidade no desempenho por núcleo: ao escalar para 12 *threads*, o escalonador distribui tarefas para os E-cores, que possuem menor desempenho individual por ciclo, desbalanceando a carga. Ambos os fenômenos são consistentes com as previsões da Lei de Amdahl aplicada a sistemas com recursos compartilhados limitados [Amdahl 1967].

5.2. Hierarquia de Memória: Largura de Banda e Localidade

A Tabela 3 apresenta os resultados do benchmark *mbw* para os três tamanhos de array avaliados.

Tabela 3. Largura de banda de memória por tamanho de array

Tamanho	Vazão (MiB/s)	Nível operante
16 MiB	6.900	L3 / RAM
128 MiB	7.370	RAM
1.024 MiB	7.363	RAM

A ausência de queda de desempenho entre os três tamanhos evidencia que o array de 16 MiB já supera o cache L3 de 12 MiB, estabelecendo a memória principal como nível operante em todos os testes. Para observar a transição entre L3 e DRAM, seria necessário testar arrays em torno de 6–10 MiB. A vazão estável em torno de 7,3 GiB/s corresponde à largura de banda sustentada por uma única *stream* sequencial de cópia — o *mbw* opera com um único fluxo de dados (*single-thread*). O teto teórico agregado do canal DDR4 é da ordem de 25,6 GB/s; atingi-lo requer múltiplos fluxos de memória concorrentes. Portanto, 7,3 GiB/s representa o limite de uma única *stream* sequencial, coerente com o regime *memory-bound* identificado nos demais experimentos.

O teste de localidade espacial registrou 4,54 s para o *loop* com acesso sequencial por linhas da matriz e 5,17 s para o *loop* com acesso por colunas, razão de 1,14x. Essa

diferença indica que o acesso não-contíguo por colunas degrada a eficácia do *prefetcher* de hardware e a localidade de cache e TLB: cada passo entre elementos adjacentes de uma coluna representa um salto de 32 KiB, impedindo a antecipação automática de linhas de cache contíguas. O custo dominante neste experimento é, contudo, o *overhead* do interpretador Python, que percorre a matriz elemento a elemento em laço puro; o efeito de localidade permanece observável mesmo parcialmente mascarado por esse custo. *TLB misses* e caminhadas na tabela de páginas (*page table walks*) são um dos fatores plausíveis da degradação, mas não constituem sua causa única nem principal.

5.3. Memória RAM, Virtual e Armazenamento

O sistema dispõe de 7,6 GiB de RAM, com 5,6 GiB disponíveis em repouso e 2,0 GiB de *swap*. Durante a pressão de 80% via `stress-ng`, o campo `so` (*swap-out*) do `vmstat` registrou valor de 22, enquanto `si` (*swap-in*) permaneceu em zero, indicando paginação de saída sem recuperação de páginas do disco no intervalo de medição.

A inspeção do processo Firefox via `/proc/PID/status` revelou `VmPeak` de 20.409.104 kB (aproximadamente 19,5 GiB) e `VmRSS` de 573.028 kB (aproximadamente 560 MB). A discrepância de 35x entre o espaço de endereçamento virtual reservado e a memória física ocupada ilustra o mecanismo de *demand paging* do Linux: o kernel mapeia páginas virtuais sem alocar frames físicos até o primeiro acesso, tornando possível que um processo reserve dezenas de gigabytes de espaço virtual em uma máquina com apenas 8 GiB de RAM instalados (7,6 GiB úteis ao sistema) [Tanenbaum and Bos 2015].

O benchmark `fio` registrou *throughput* sequencial de 1.527 MB/s e 1.918 IOPS com latência média de 517 μ s para leituras aleatórias de 4 KiB. A Tabela 4 apresenta os links PCIe identificados.

Tabela 4. Características dos links PCIe dos dispositivos identificados

Dispositivo	ID PCI	LnkCap	LnkSta
Controlador NVMe	1C5C:1D59	Gen 4x4	Gen 3x4
GPU	10DE:28A1	Gen 4x8	Gen 4x8
Controlador de rede	14C3:7961	Gen 2x1	Gen 2x1

O controlador NVMe opera em Gen 3x4 apesar de possuir capacidade para Gen 4x4, limitando a largura de banda bruta máxima de 8 GB/s (teórico Gen 4x4) para 3,94 GB/s brutos (3,5 GB/s úteis, teórico Gen 3x4). O *throughput* sequencial medido pelo `fio` de 1.527 MB/s é substancialmente inferior a esse limite, o que indica que o link PCIe Gen 3x4 *não* é o gargalo do I/O sequencial observado — o fator limitante é a camada de virtualização do WSL2, que interpõe uma chamada ao hipervisor a cada operação de bloco. O resultado aponta subutilização da capacidade de interconexão do dispositivo; recomenda-se auditar a configuração do link na BIOS para verificar a possibilidade de habilitação do modo Gen 4x4.

5.4. Interrupções e I/O

A contagem de interrupções antes e após a transferência de 2 GB via `dd` registrou incrementos de +2.054 na linha `CAL` (*Inter-Processor Interrupts*) e +2.008 na linha `HYP` (*Hypervisor callback interrupts*). A correspondência entre os 2.008 *callbacks* e os 2.000

blocos de 1 MiB transferidos — constatada por medição direta neste trabalho por meio do contador `/proc/interrupts` [Love 2010] — confirma que cada conclusão de bloco gerou uma notificação separada do hipervisor Hyper-V para o kernel Linux convidado. Esse comportamento é consequência estrutural da virtualização de dispositivos no WSL2 [Tanenbaum and Bos 2015]: o armazenamento é mediado pela camada Hyper-V, que interpõe um *callback* a cada operação de I/O completada, replicando para o sistema operacional convidado a semântica de interrupção de um controlador NVMe físico. A conclusão sobre o overhead de I/O no WSL2 apoia-se, portanto, na evidência medida neste trabalho, não em fontes externas.

5.5. Workload Integrado e Identificação do Gargalo

O script de carga sintética integrada registrou 89.111,3 Mop/s na fase CPU-bound, 4,29 GB/s na fase *memory-bound* e 1.192,1 MB/s na fase I/O-bound. A saturação de CPU foi de zero núcleos acima de 50% durante a fase de medição, confirmando que o processador permaneceu ocioso aguardando dados da memória principal — comportamento diagnóstico de um sistema *memory-bound* [Williams et al. 2009].

A largura de banda de 4,29 GB/s medida na fase integrada é inferior à registrada pelo *mbw* (aproximadamente 7,3 GiB/s). A diferença é atribuída ao fato de que o *mbw* utiliza blocos de cópia otimizados para saturar o *prefetcher* de hardware, ao passo que o workload integrado realiza uma única cópia sequencial sem otimizações específicas. Os três conjuntos de evidências — teste de paralelismo, *mbw* e workload integrado — convergem para o mesmo gargalo.

6. Considerações Finais

Este trabalho demonstrou experimentalmente que o gargalo primário da plataforma analisada é a largura de banda da memória principal, identificado pela degradação de desempenho ao escalar o paralelismo além de quatro *threads*, pela estabilidade da vazão do *mbw* em todos os tamanhos de array testados e pela ociosidade de CPU durante a fase *memory-bound* do workload integrado. A análise dos barramentos PCIe revelou assimetria entre a capacidade declarada do controlador NVMe (Gen 4x4) e o link efetivamente negociado (Gen 3x4), representando uma oportunidade de ganho de desempenho em I/O sequencial. O estudo da memória virtual ilustrou o funcionamento do mecanismo de *demand paging* por meio da discrepância de 35x entre *VmPeak* e *VmRSS* do processo Firefox.

Como trabalhos futuros, propõe-se a repetição dos experimentos com arrays menores que o cache L3 para caracterizar a curva completa de transição entre os níveis da hierarquia de memória, a avaliação do impacto do escalonamento de tarefas entre P-cores e E-cores em cargas heterogêneas e a comparação das medições em WSL2 com execução nativa em Linux, isolando o efeito da camada de virtualização sobre os subsistemas de memória e I/O.

A. Artefatos do Repositório

Os artefatos brutos que sustentam todos os resultados apresentados neste artigo encontram-se disponíveis em repositório público, garantindo a reprodutibilidade dos experimentos:

<https://github.com/Pedro-Coutinho2612/infra-hw-artigo>

Saídas de medição (diretório dados/):

- `info_cpu.txt` — saída completa do `lscpu` com identificação do processador
- `cinebench_r23.txt` — resultados do Cinebench R23 (*single* e *multi core*)
- `saida_paralelismo.txt` — resultados do teste de escalabilidade de *threads*
- `saida_localidade.txt` — resultados do teste de localidade espacial (linhas vs. colunas)
- `saida_workload.txt` — resultados do workload integrado (fases CPU, memória e I/O)
- `medicoes_mbw.txt` — saída do `mbw` para arrays de 16, 128 e 1.024 MiB
- `medicoes_fio.txt` — saída do `fio` (sequencial 1 MiB e aleatório 4 KiB)
- `medicoes_memoria.txt` — saída de `free`, `vmstat` e `/proc/PID/status`
- `medicoes_pcie.txt` — links PCIe (`LnkCap/LnkSta`) via `lspci -vv`
- `medicoes_interrupcoes.txt` — diferença de `/proc/interrupts` antes/após `dd`

Scripts Python (diretório scripts/):

- `teste_paralelismo.py` — varia o número de processos (1, 2, 4, máx.) e mede *speedup*
- `teste_localidade.py` — compara acesso por linhas e por colunas em matriz NumPy
- `workload_completo.py` — três fases (*CPU-bound*, *memory-bound*, *I/O-bound*) com monitoramento por núcleo via `psutil`

Referências

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the Spring Joint Computer Conference (AFIPS)*, pages 483–485, New York, NY. ACM.
- Budruk, R., Anderson, D., and Shanley, T. (2003). *PCI Express System Architecture*. Addison-Wesley, Boston, MA.
- Diamond, J., Burtscher, M., McCalpin, J. D., Kim, B.-D., Keckler, S. W., and Browne, J. C. (2011). Evaluation and optimization of multicore performance bottlenecks in supercomputing applications. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 32–43, Austin, TX. IEEE.
- Hennessy, J. L. and Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, Cambridge, MA, 6 edition.
- Intel Corporation (2021). Intel performance hybrid architecture and software optimizations, part one: Introduction. White Paper 349168, Intel Corporation. Disponível em: https://cdrdv2-public.intel.com/685861/211115_Hybrid_WP_1_Introduction_v1.2.pdf.
- Intel Corporation (2022a). 13th generation intel core processors: Datasheet, volume 1. Technical report, Intel Corporation. Disponível em: <https://www.intel.com/content/www/us/en/products/docs/processors/core/13th-gen-core-datasheet-vol-1.html>.

- Intel Corporation (2022b). Intel thread director: Delivering optimal performance for hybrid architecture. Technical report, Intel Corporation. Disponível em: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-thread-director.html>.
- Love, R. (2010). *Linux Kernel Development*. Addison-Wesley, Upper Saddle River, NJ, 3 edition.
- McCalpin, J. D. (1995). STREAM: Sustainable memory bandwidth in high performance computers. Technical report, University of Virginia. Disponível em: <https://www.cs.virginia.edu/stream/>.
- McVoy, L. and Staelin, C. (1996). Imbench: Portable tools for performance analysis. In *Proceedings of the USENIX Annual Technical Conference*, pages 279–294, San Diego, CA. USENIX Association.
- Patterson, D. A. and Hennessy, J. L. (2021). *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, Cambridge, MA, 2 edition.
- Rogers, B. M., Krishna, A., Bell, G. B., Vu, K., Jiang, X., and Solihin, Y. (2009). Scaling the bandwidth wall: Challenges in and avenues for CMP scaling. In *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA)*, pages 371–382, New York, NY. ACM.
- Tanenbaum, A. S. and Bos, H. (2015). *Organização Estruturada de Computadores*. Pearson, São Paulo, 6 edition.
- Williams, S., Waterman, A., and Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76.